Technical Documentation

Objective

The objective of this document is to provide a detailed explanation of the RAG-based Customer Support Assistant system, including its architecture, design decisions, workflow, and implementation logic.

1. Introduction

What is RAG?

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with language models. Instead of relying only on pre-trained knowledge, the system retrieves relevant information from a knowledge base and uses it to generate accurate responses.

Why it is Needed

- Reduces hallucinations in AI responses
- Provides context-based answers
- Improves accuracy using real data
- Enables domain-specific support systems

Use Case Overview

This system is designed as a **Customer Support Assistant** that:

- Answers user queries using a knowledge base
- Retrieves relevant information using embeddings
- Uses a workflow engine for decision-making
- Escalates queries to human support when needed

2. System Architecture Explanation

The system follows a modular architecture:

- User Interface (Streamlit) → accepts user queries
- Document Ingestion Pipeline → loads and processes knowledge base
- Embedding System → converts text into vector form
- Vector Database (ChromaDB) → stores embeddings
- Retriever → fetches relevant data
- LLM Layer → generates final answer
- LangGraph Workflow → manages control flow
- HITL Module → handles escalation

Component Interactions

1. User submits query via UI
2. Query is passed to LangGraph
3. Retriever fetches relevant chunks from ChromaDB
4. Decision logic evaluates context
5. If relevant → LLM generates answer
6. If not → routed to HITL
7. Final response returned to user

3. Design Decisions

Chunk Size Choice

- Chunk size: ~500 characters
- Reason: balances context and retrieval accuracy

Embedding Strategy

- Model: all-MiniLM-L6-v2
- Reason:
    - Lightweight
    - Fast
    - Good semantic understanding

Retrieval Approach

- Top-k retrieval (k = 5)
- Additional filtering based on query keywords
- Returns best 1–2 results

Prompt Design Logic

- Uses retrieved context directly
- Avoids unnecessary generation
- Ensures grounded responses

4. Workflow Explanation

LangGraph Usage

LangGraph is used to define the workflow of the system, including how queries are processed and routed.

Node Responsibilities

- **Processing Node** → retrieves and filters relevant data

- **Output Node** → generates final answer
- **HITL Node** → handles escalation

State Transitions

- Query → Context → Answer

5. Conditional Logic

Intent Detection

- Based on keywords in the query
- Identifies user intent (refund, contact, etc.)

Routing Decisions

- If relevant context exists → generate answer
- If no context → escalate to HITL

6. HITL Implementation

Role of Human Intervention

- Handles cases where system cannot answer
- Ensures reliability and correctness

Benefits

- Improves accuracy
- Handles complex queries
- Builds user trust

Limitations

- Requires human availability
- Slower response time

7. Challenges & Trade-offs

Retrieval Accuracy vs Speed

- Higher k → better accuracy but slower
- Lower k → faster but less accurate

Chunk Size vs Context Quality

- Smaller chunks → precise but limited context
- Larger chunks → more context but less precise

Cost vs Performance

- Lightweight models reduce cost

- Larger models improve performance

8. Testing Strategy

Testing Approach

- Tested with multiple user queries
- Verified retrieval correctness
- Checked HITL triggering

Sample Queries

- "refund policy"
- "contact support"
- "password reset"

9. Future Enhancements

- Multi-document support
- Feedback-based learning
- Memory integration
- Deployment on cloud
- UI improvements